

Storage Engine and Block Cache

 This is a legacy version of the Mini-LSM course and we will not maintain it anymore. We now have a better version of this course and this chapter is now part of [Mini-LSM Week 1 Day 5: Read Path](#) and [Mini-LSM Week 1 Day 6: Write Path](#)

- [Task 1 - Put and Delete](#)
- [Task 2 - Get](#)
- [Task 3 - Scan](#)
- [Task 4 - Sync](#)
- [Task 5 - Block Cache](#)
- [Extra Tasks](#)

In this part, you will need to modify:

- `src/lsm_iterator.rs`
- `src/lsm_storage.rs`
- `src/table.rs`
- Other parts that use `SsTable::read_block`

You can use `cargo x copy-test day4` to copy our provided test cases to the starter code directory. After you have finished this part, use `cargo x scheck` to check the style and run all test cases. If you want to write your own test cases, write a new module `#[cfg(test)] mod user_tests { /* your test cases */ }` in `table.rs`. Remember to remove `#![allow(...)]` at the top of the modules you modified so that cargo clippy can actually check the styles.

Task 1 - Put and Delete

Before implementing put and delete, let's revisit how LSM tree works. The structure of LSM includes:

- Mem-table: one active mutable mem-table and multiple immutable mem-tables.
- Write-ahead log: each mem-table corresponds to a WAL.
- SSTs: mem-table can be flushed to the disk in SST format. SSTs are organized in multiple levels.

In this part, we only need to take the lock, write the entry (or tombstone) into the active mem-table. You can modify `lsm_storage.rs`.

Task 2 - Get

To get a value from the LSM, we can simply probe from active memtable, immutable memtables (from latest to earliest), and all the SSTs. To reduce the critical section, we can hold the read lock to copy all the pointers to mem-tables and SSTs out of the `LsmStorageInner` structure, and create iterators out of the critical section. Be careful about the order when creating iterators and probing.

Task 3 - Scan

To create a scan iterator `LsmIterator`, you will need to use `TwoMergeIterator` to merge `MergeIterator` on mem-table and `MergeIterator` on SST. You can implement this in `lsm_iterator.rs`. Optionally, you can implement `FusedIterator` so that if a user accidentally calls `next` after the iterator becomes invalid, the underlying iterator won't panic.

The sequence of key-value pairs produced by `TwoMergeIterator` may contain empty value, which means that the value is deleted. `LsmIterator` should filter these empty values. Also it needs to correctly handle the start and end bounds.

Task 4 - Sync

In this part, we will implement mem-tables and flush to L0 SSTs in `lsm_storage.rs`. As in task 1, write operations go directly into the active mutable mem-table. Once `sync` is called, we flush SSTs to the disk in two steps:

- Firstly, move the current mutable mem-table to immutable mem-table list, so that no future requests will go into the current mem-table. Create a new mem-table. All of these should happen in one single critical section and stall all reads.
- Then, we can flush the mem-table to disk as an SST file without holding any lock.
- Finally, in one critical section, remove the mem-table and put the SST into `l0_tables`.

Only one thread can sync at a time, and therefore you should use a mutex to ensure this requirement.

Task 5 - Block Cache

Now that we have implemented the LSM structure, we can start writing something to the disk! Previously in `table.rs`, we implemented a `FileObject` struct, without writing

anything to disk. In this task, we will change the implementation so that:

- `read` will read from the disk without any caching using `read_exact_at` in `std::os::unix::fs::FileExt`.
- The size of the file should be stored inside the struct, and `size` function directly returns it.
- `create` should write the file to the disk. Generally you should call `fsync` on that file. But this would slow down unit tests a lot. Therefore, we don't do `fsync` until day 6 recovery.
- `open` remains unimplemented until day 6 recovery.

After that, we can implement a new `read_block_cached` function on `SsTable` so that we can leverage block cache to serve read requests. Upon initializing the `LsmStorage` struct, you should create a block cache of 4GB size using `moka-rs`. Blocks are cached by SST id + block id. Use `try_get_with` to get the block from cache / populate the cache if cache miss. If there are multiple requests reading the same block and cache misses, `try_get_with` will only issue a single read request to the disk and broadcast the result to all requests.

Remember to change `SsTableIterator` to use the block cache.

Extra Tasks

- As you might have seen, each time we do a get, put or deletion, we will need to take a read lock protecting the LSM structure; and if we want to flush, we will need to take a write lock. This can cause a lot of problems. Some lock implementations are fair, which means as long as there is a writer waiting on the lock, no reader can take the lock. Therefore, the writer will wait until the slowest reader finishes its operation before it can actually do some work. One possible optimization is to implement `WriteBatch`. We don't need to immediately write users' requests into mem-table + WAL. We can allow users to do a batch of writes.
- Align blocks to 4K and use direct I/O.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.